

6장 단답형 40문항

1. 업캐스팅과 다운캐스팅

1. 서브 클래스 객체를 슈퍼 클래스 타입으로 변환하는 것을 무엇이라고 하는가?
2. 슈퍼 클래스 타입의 레퍼런스를 서브 클래스 타입으로 변환하는 것을 무엇이라고 하는가?
3. 업캐스팅은 명시적 타입 변환 없이 자동으로 이루어질 수 있는가? 가능 또는 불가능 으로 쓰시오.
4. 다운캐스팅을 할 때 필요한 타입 변환 방식을 쓰시오.
5. 다음 코드에서 발생하는 타입 변환을 쓰시오.

```
class Person { }  
class Student extends Person { }  
  
Student s = new Student();  
Person p = s;
```

6. 다음 코드의 빈칸을 채우시오.

```
Person p = new Student();  
Student s = (_____)p;
```

7. 업캐스팅된 레퍼런스로 직접 접근할 수 있는 것은 슈퍼 클래스와 서브 클래스 중 어느 클래스의 멤버인가?
8. 다음 코드에서 레퍼런스 변수 p의 선언 타입을 쓰시오.

```
Person p = new Student();
```

9. 위 코드에서 p가 실제로 가리키는 객체의 타입을 쓰시오.
10. 다음 코드에서 발생하는 타입 변환을 쓰시오.

```
Person p = new Student();  
Student s = (Student)p;
```

II. instanceof와 객체 타입 판별

11. 레퍼런스가 가리키는 객체의 타입을 식별하기 위해 사용하는 연산자를 쓰시오.

12. `instanceof` 연산 결과의 자료형을 쓰시오.

13. 다음 코드의 실행 결과를 쓰시오.

```
class Person { }  
class Student extends Person { }  
  
Person p = new Student();  
System.out.println(p instanceof Student);
```

14. 다음 코드의 실행 결과를 쓰시오.

```
Person p = new Student();  
System.out.println(p instanceof Person);
```

15. 다음 코드 실행 후 출력되는 문자열을 쓰시오.

```
Person p = new Student();  
  
if (p instanceof Student)  
    System.out.print("Student");
```

16. 다음 상속 관계에서 `Professor` 객체는 `Researcher` 타입이기도 한가? `true` 또는 `false` 로 쓰시오.

```
class Person { }  
class Researcher extends Person { }  
class Professor extends Researcher { }
```

17. 위 상속 관계에서 `Professor` 객체는 `Person` 타입이기도 한가? `true` 또는 `false` 로 쓰시오.

18. 다음 코드의 실행 결과를 쓰시오.

```
Person p = new Person();  
System.out.println(p instanceof Person);
```

III. 메소드 오버라이딩

19. 슈퍼 클래스의 메소드를 서브 클래스에서 재정의하는 것을 무엇이라고 하는가?
20. 메소드 오버라이딩 시 슈퍼 클래스와 서브 클래스의 메소드 이름은 같아야 하는가? **같다** 또는 **다르다** 로 쓰시오.
21. 메소드 오버라이딩 시 매개변수의 타입과 개수는 슈퍼 클래스의 메소드와 같아야 하는가? **같다** 또는 **다르다** 로 쓰시오.
22. 메소드 오버라이딩에 의해 실행할 메소드가 실행 시간에 결정되는 것을 무엇이라고 하는가?
23. 다음 코드의 실행 결과를 쓰시오.

```
class Shape {
    public void draw() {
        System.out.println("Shape");
    }
}

class Line extends Shape {
    public void draw() {
        System.out.println("Line");
    }
}

Line line = new Line();
line.draw();
```

24. 다음 코드의 실행 결과를 쓰시오.

```
Shape shape = new Line();
shape.draw();
```

25. 다음 코드에서 **Shape shape = new Line();** 에 발생하는 타입 변환을 쓰시오.

```
Shape shape = new Line();
```

26. 다음 코드의 실행 결과를 쓰시오.

```
class Shape {
    public void draw() {
        System.out.println("Shape");
    }
}
```

```

}

class Circle extends Shape {
    public void draw() {
        System.out.println("Circle");
    }
}

Shape s = new Circle();
s.draw();

```

27. 메소드 오버라이딩을 통해 실현되는 객체 지향의 특성을 쓰시오.
28. 메소드 오버라이딩은 컴파일 시간과 실행 시간 중 어느 시점의 다형성을 실현하는가?
29. 다음 코드에서 `paint(new Rect());` 가 호출되었을 때 출력되는 문자열을 쓰시오. 단, Rect 클래스는 `draw()` 를 오버라이딩하여 `"Rect"` 를 출력한다.

```

static void paint(Shape p) {
    p.draw();
}

```

30. 다음 코드의 출력 순서를 쓰시오.

```

paint(new Shape());
paint(new Line());
paint(new Rect());
paint(new Circle());

```

각 클래스의 `draw()` 는 각각 클래스 이름인 `Shape`, `Line`, `Rect`, `Circle` 을 출력한다.

IV. 동적 바인딩과 super

31. 메소드를 실행 시점(run time)에 결정하는 것을 무엇이라고 하는가?
32. 동적 바인딩에서는 슈퍼 클래스의 메소드와 서브 클래스에서 오버라이딩된 메소드 중 어느 메소드가 호출되는가?
33. 서브 클래스에서 슈퍼 클래스의 멤버에 접근할 때 사용하는 키워드를 쓰시오.
34. `super` 는 슈퍼 클래스와 서브 클래스 중 어느 클래스의 멤버에 접근하기 위해 사용하는가?
35. 다음 코드의 실행 결과를 쓰시오.

```

class Shape {
    public void draw() {
        System.out.println("Shape");
    }
}

class Circle extends Shape {
    public void draw() {
        super.draw();
    }
}

Circle c = new Circle();
c.draw();

```

36. `super` 를 통한 슈퍼 클래스 멤버 접근은 정적 바인딩과 동적 바인딩 중 어느 방식으로 처리되는가?

V. 오버라이딩과 오버로딩

37. 같은 이름의 메소드를 매개변수의 개수나 타입을 다르게 하여 여러 개 작성하는 것을 무엇이라고 하는가?
38. 메소드 오버로딩은 컴파일 시간과 실행 시간 중 어느 시점의 다형성을 실현하는가?
39. 오버라이딩과 오버로딩 중 상속 관계에서 슈퍼 클래스의 메소드를 서브 클래스에서 재 정의하는 것은 무엇인가?
40. 다음 코드의 실행 결과를 쓰시오.

```

class Weapon {
    protected int fire() {
        return 1;
    }
}

class Cannon extends Weapon {
    protected int fire() {
        return 10;
    }
}

Weapon weapon = new Cannon();
System.out.println(weapon.fire());

```

6장 단답형 40문항 정답 및 해설

문항	정답	해설
1	업캐스팅 (upcasting)	서브 클래스 객체를 슈퍼 클래스 타입으로 변환하는 것이다.
2	다운캐스팅 (downcasting)	슈퍼 클래스 타입의 레퍼런스를 서브 클래스 타입으로 변환하는 것이다.
3	가능	업캐스팅은 별도의 명시적 타입 변환 없이 자동으로 이루어질 수 있다.
4	명시적 타입 변환	다운캐스팅은 <code>(Student)</code> 와 같이 변환할 타입을 직접 명시해야 한다.
5	업캐스팅	<code>Student</code> 객체를 <code>Person</code> 타입의 레퍼런스에 대입했으므로 업캐스팅이다.
6	Student	<code>Student s = (Student)p;</code> 와 같이 작성하여 다운캐스팅한다.
7	슈퍼 클래스의 멤버	업캐스팅된 레퍼런스로는 슈퍼 클래스에 선언된 멤버에 접근할 수 있다.
8	Person	<code>p</code> 는 <code>Person</code> 타입으로 선언된 레퍼런스 변수이다.
9	Student	레퍼런스의 선언 타입은 <code>Person</code> 이지만 실제 생성된 객체는 <code>new Student()</code> 이므로 <code>Student</code> 객체이다.
10	다운캐스팅	<code>(Student)p</code> 와 같이 슈퍼 클래스 타입의 레퍼런스를 서브 클래스 타입으로 명시적으로 변환한다.

업캐스팅은 서브 클래스 객체를 슈퍼 클래스 타입으로 변환하며 자동 타입 변환이 가능하고, 다운캐스팅은 명시적인 타입 변환이 필요하다.

문항	정답	해설
11	instanceof	레퍼런스가 가리키는 객체의 타입을 식별할 때 사용하는 연산자이다.
12	boolean	<code>instanceof</code> 의 결과는 <code>true</code> 또는 <code>false</code> 이다.
13	true	<code>p</code> 가 실제로 <code>Student</code> 객체를 가리키므로 <code>p instanceof Student</code> 는 true이다.
14	true	<code>Student</code> 는 <code>Person</code> 을 상속하므로 <code>Student</code> 객체는 <code>Person</code> 타입이기도 하다.
15	Student	조건식 <code>p instanceof Student</code> 가 true이므로 <code>"Student"</code> 가 출력된다.

문항	정답	해설
16	true	Professor가 Researcher를 상속하므로 Professor 객체는 Researcher 타입이기도 하다.
17	true	Researcher가 Person을 상속하고 Professor가 Researcher를 상속하므로 Professor 객체는 Person 타입에도 해당한다.
18	true	p 가 실제 Person 객체를 가리키므로 p instanceof Person 은 true이다.

instanceof 는 객체레퍼런스 instanceof 클래스타입 형태로 사용하며 결과는 true 또는 false이다. 상속 관계에서는 서브 클래스 객체가 상위 클래스 타입에도 해당한다. 자료의 예에서도 Professor 객체는 Person, Researcher, Professor 타입에 모두 해당한다.

문항	정답	해설
19	메소드 오버라이딩(Method Overriding)	슈퍼 클래스에 정의된 메소드를 서브 클래스에서 다시 정의하는 것이다.
20	같다	오버라이딩하려는 메소드의 이름은 슈퍼 클래스 메소드와 동일해야 한다.
21	같다	매개변수의 타입과 개수도 동일해야 한다.
22	동적 바인딩(dynamic binding)	오버라이딩된 메소드 중 실제로 실행할 메소드를 실행 시간에 결정한다.
23	Line	line 은 Line 객체이므로 Line에서 오버라이딩한 draw() 가 실행된다.
24	Line	레퍼런스 타입은 Shape이지만 실제 객체가 Line이므로 Line의 draw() 가 호출된다.
25	업캐스팅	Line 객체를 슈퍼 클래스인 Shape 타입의 레퍼런스에 대입하였다.
26	Circle	실제 객체가 Circle이므로 Circle에서 오버라이딩한 draw() 가 실행된다.
27	다형성(Polymorphism)	동일한 이름의 메소드가 객체에 따라 서로 다른 동작을 수행하게 할 수 있다.
28	실행 시간(run time)	오버라이딩은 동적 바인딩을 통해 실행 시간 다형성을 실현한다.
29	Rect	전달된 실제 객체가 Rect이므로 Rect의 draw() 가 호출된다.

문항	정답	해설
30	Shape → Line → Rect → Circle	각 레퍼런스가 실제로 가리키는 객체의 오버라이딩된 <code>draw()</code> 가 순서대로 실행된다.

메소드 오버라이딩은 슈퍼 클래스의 메소드를 서브 클래스에서 재정의하는 것으로, 메소드 이름·매개변수 타입과 개수·리턴 타입 등을 동일하게 작성한다. 이때 동적 바인딩이 발생한다. 자료의 `paint()` 예제에서도 실제 객체에 따라 `Shape`, `Line`, `Rect`, `Circle` 의 `draw()` 가 각각 호출된다.

문항	정답	해설
31	동적 바인딩 (dynamic binding)	실행할 메소드를 실행 시간에 결정하는 방식이다.
32	서브 클래스에서 오버라이딩된 메소드	실제 객체에 오버라이딩된 메소드가 있으면 해당 메소드가 호출된다.
33	super	서브 클래스에서 슈퍼 클래스의 멤버에 접근할 때 사용한다.
34	슈퍼 클래스	<code>super</code> 는 슈퍼 클래스의 멤버를 접근하기 위한 레퍼런스이다.
35	Shape	<code>super.draw()</code> 는 슈퍼 클래스 Shape의 <code>draw()</code> 를 직접 호출하므로 <code>"Shape"</code> 이 출력된다.
36	정적 바인딩 (static binding)	<code>super</code> 를 이용한 슈퍼 클래스 멤버 접근은 컴파일러가 정적 바인딩으로 처리한다.

동적 바인딩은 실행할 메소드를 실행 시간에 결정하고 오버라이딩된 메소드가 호출되는 방식이다. 반면 `super` 는 슈퍼 클래스의 멤버를 접근할 때 사용하며 `super` 를 통한 접근은 정적 바인딩으로 처리된다.

문항	정답	해설
37	메소드 오버로딩 (Method Overloading)	같은 이름의 메소드를 매개변수의 개수나 타입을 다르게 하여 중복 작성하는 것이다.
38	컴파일 시간(compile time)	오버로딩은 컴파일 시 호출할 메소드가 결정되는 컴파일 타임 다형성이다.
39	메소드 오버라이딩	상속 관계에서 슈퍼 클래스의 메소드를 서브 클래스가 재정의하는 것이다.
40	10	<code>weapon</code> 의 선언 타입은 <code>Weapon</code> 이지만 실제 객체는 <code>Cannon</code> 이다. <code>Cannon</code> 의 <code>fire()</code> 가 오버라이딩되

문항	정답	해설
		어 있으므로 동적 바인딩에 의해 10을 반환한다.

오버라이딩은 **실행 시간 다형성**, 오버로딩은 **컴파일 시간 다형성**을 실현한다. 40번과 동일한 Weapon/Cannon 예제에서도 Weapon의 `fire()` 는 1, Cannon에서 오버라이딩한 `fire()` 는 10을 반환한다.